

UML. Use Case Diagram



Use Case Diagram

The use case diagram model describes the functional purpose of the system.

A use case diagram is an initial conceptual representation or conceptual model of a system during its design and development.

A use case diagram is a diagram that depicts the relationships between actors and use cases.

Use Case Diagram

Creating a use case diagram has the following objectives:

- ✓ Define the general boundaries and context of the domain being modeled during the initial stages of system design
- ✓ *Formulate general requirements for the functional behavior of the system under design*
- ✓ Develop an initial conceptual model of the system for subsequent elaboration into logical and physical models
- ✓ *Prepare the source documentation to facilitate interaction between the system developers and its clients and users*

Use Case Diagram

A use case diagram is a diagram that shows a set of use cases and actors, as well as the relationships between them

Use case diagrams are used to model the system from the perspective of its use cases

A use case diagram includes :

- ✓ use cases
- ✓ actors
- ✓ dependency, generalization, and association relationships

Defining the use case



Прецедент

A diagram showing a light pink oval with a black border and a drop shadow. The word 'Прецедент' is written inside in black. A thin black line extends from the bottom of the oval towards the 'Актор' icon.

A use case is a description of a set of action sequences (including their variations) performed by a system to enable an actor to achieve a specific result.



Actor — a logically related set of roles played by use-case users during their interaction with the use cases.

The significance of the use case for requirements analysis :

- ✓ A use case describes a set of sequences, each representing an interaction between actors and the system
- ✓ *Such interactions are system-level functions that can be used to specify, design, and document the desired system behavior during the requirements gathering and analysis phase*
- ✓ A use case defines the functional requirements for the system as a whole

Significance of the use case for system development

- ✓ Use cases, as descriptions of system-level functions, can serve as a basis for managing system development and testing
- ✓ Use cases describe system behavior and can be used as a basis for testing system components

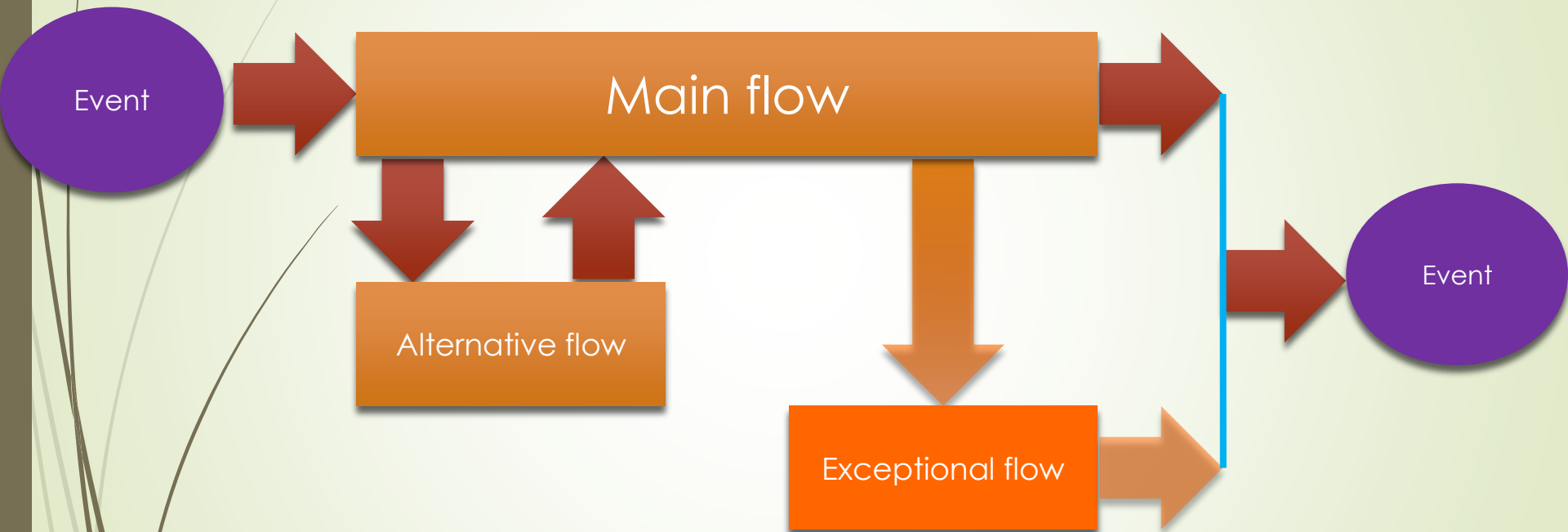
What a use case describes?

- A use case describes what the system does but does not specify how it does it

Specifying use case behavior

- The behavior of a use case can be described by specifying flows of events
- In the use case description, it is important to specify how and when it begins and ends, when it interacts with actors, and what objects are exchanged with them
- When specifying a use case, it is important to remember that it represents a description of a SET of sequences of actions

Defining the use case



Event

Event

- ✓ Each use case flow begins with an event and ends with an event
- ✓ An event can be composite - that is, the result of a logical combination of several events
- ✓ Each flow can have only one event that initiates its execution and only one event that concludes it

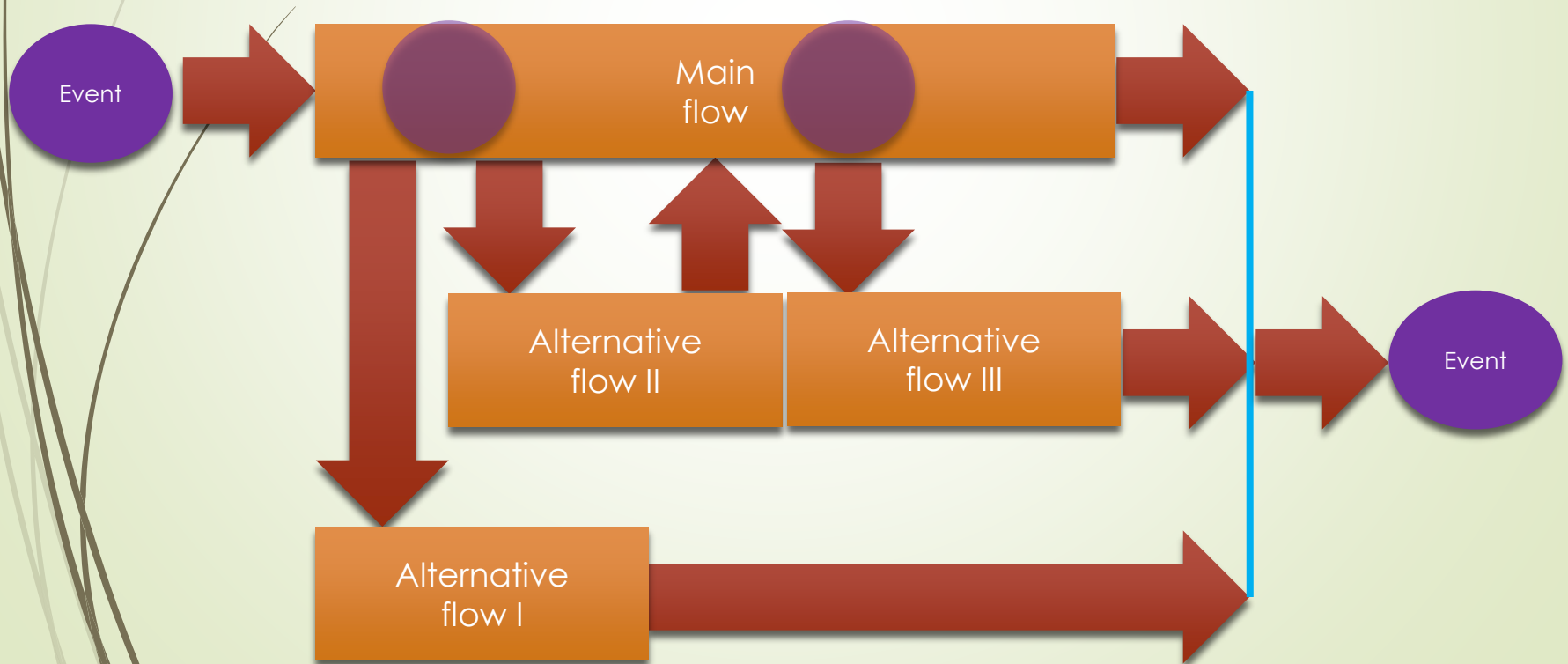
The main flow is a sequence of events in the interaction between the actor and the system that leads the actor to the use case goal

The main flow may contain branches that execute in parallel (for example, when a use case describes the interaction of multiple actors with the system)



Alternative flow

An alternative flow is a sequence of events initiated by the occurrence of a specific event within the main flow.



Exceptional flow

An exception flow is a flow initiated when the "rules of the game" of the main or alternative flows are violated during the course of a use case

An exception flow is triggered by an event that is not part of either the main flow or the alternative flows. The purpose of an exception flow is to return the execution of the use case to the normal path (main or alternative flows) or to terminate its execution gracefully

An example of triggering an exceptional flow is the processing of invalid input data.

Description of flows

- The flow of events in a use case can be described in various ways. It may take the form of informal, unstructured text or formal, structured text (including pre- and post-conditions). The only requirement for the flow description is that it must provide an unambiguous representation of the interaction between the system and the actor within the context of the use case being described
- *The description of the flows must clearly indicate at what point and under what condition alternative flows may be triggered, and at what point and under what condition an alternative or exception flow merges back into the main flow*
- For an exception flow, it must be clear at what point in the main or alternative flow it can start, and under what condition

A scenario is a sequence of actions illustrating the behavior of a system.

A use case describes several possible scenarios, each representing one of the possible combinations of use case flows.

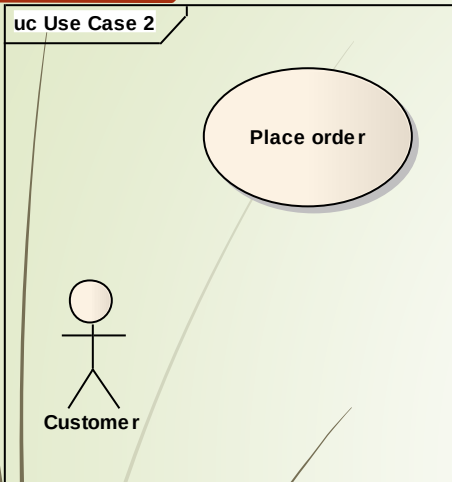
In relation to a use case, a scenario represents an instance of it.

A relatively complex system may contain several dozen use cases, each of which can unfold into several dozen scenarios.

For any use case, one can identify primary scenarios describing the most important sequences, and auxiliary scenarios describing alternative sequences.

Organizing Use Cases

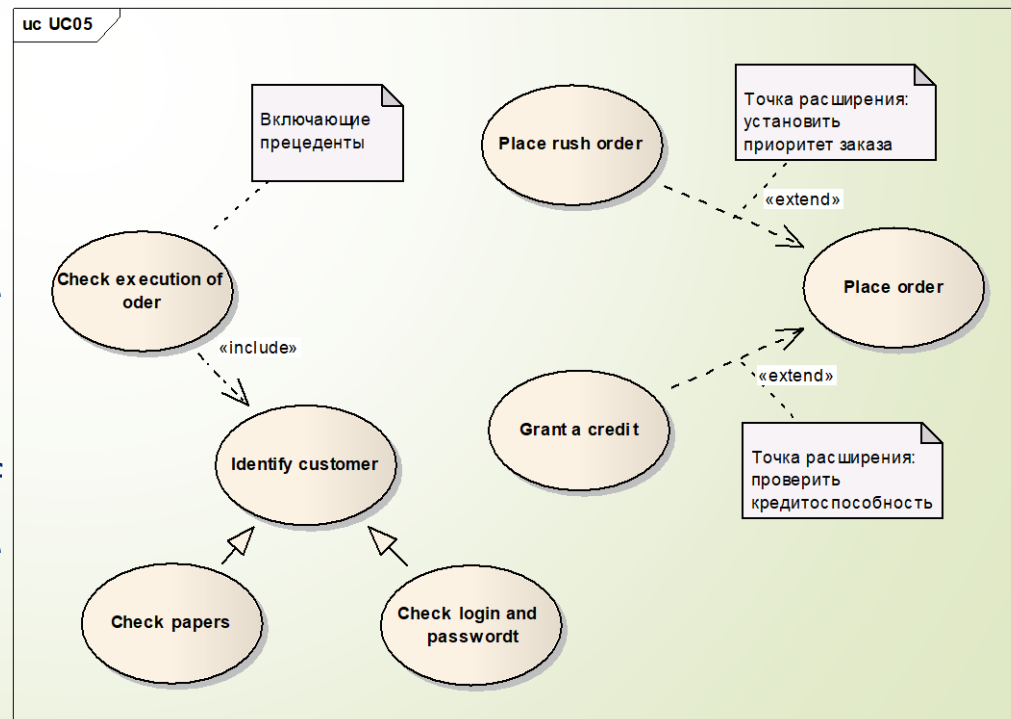
uc Use Case 2



➤ An actor can interact with a use case only via an association relationship. In this case, the association indicates that the actor communicates with the use case by sending or receiving messages

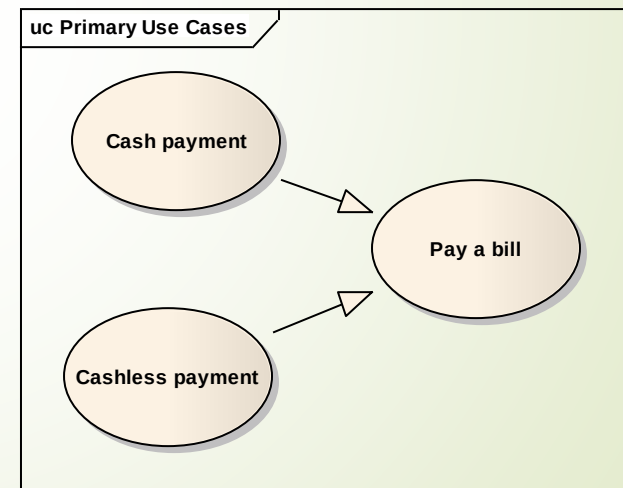
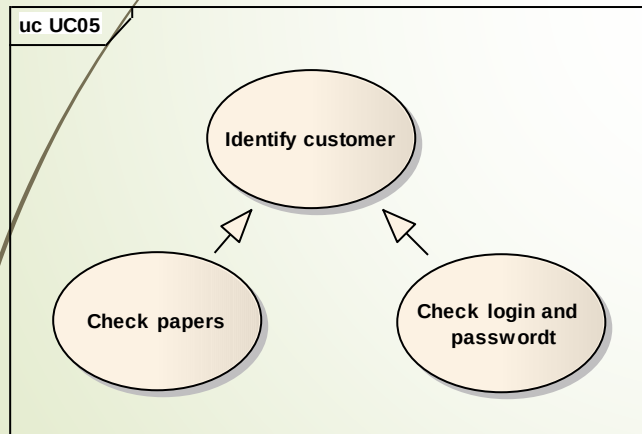
- Only generalization and dependency relationships are possible between use cases
- Only two types of dependency relationships are permitted: include and extend

uc UC05



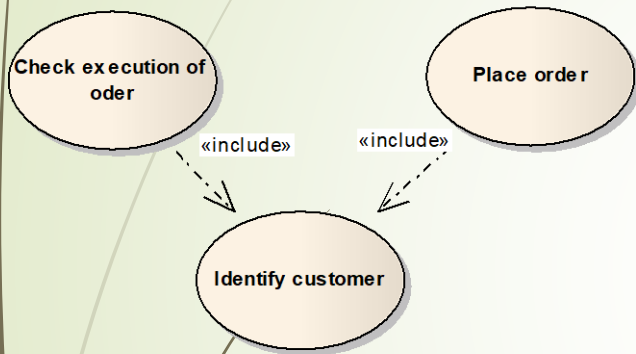
Organizing Use Cases

- A generalization relationship implies that a child use case inherits the behavior and semantics of its base use case, can override or extend that behavior, and can be substituted wherever the parent use case appears.



Organizing Use Cases

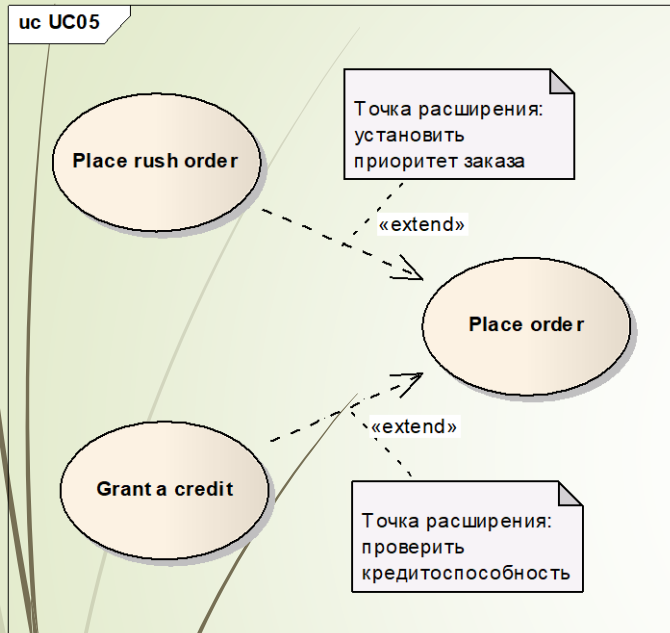
uc UC05



➤ The inclusion relationship (a dependency relationship with the «include» stereotype) between use cases signifies that, at a certain point, one use case incorporates the behavior defined in another use case.

- The inclusion relationship is used to eliminate the need to describe the same flow of events multiple times
- An included use case never exists in isolation but is part of a base use case

Organizing Use Cases

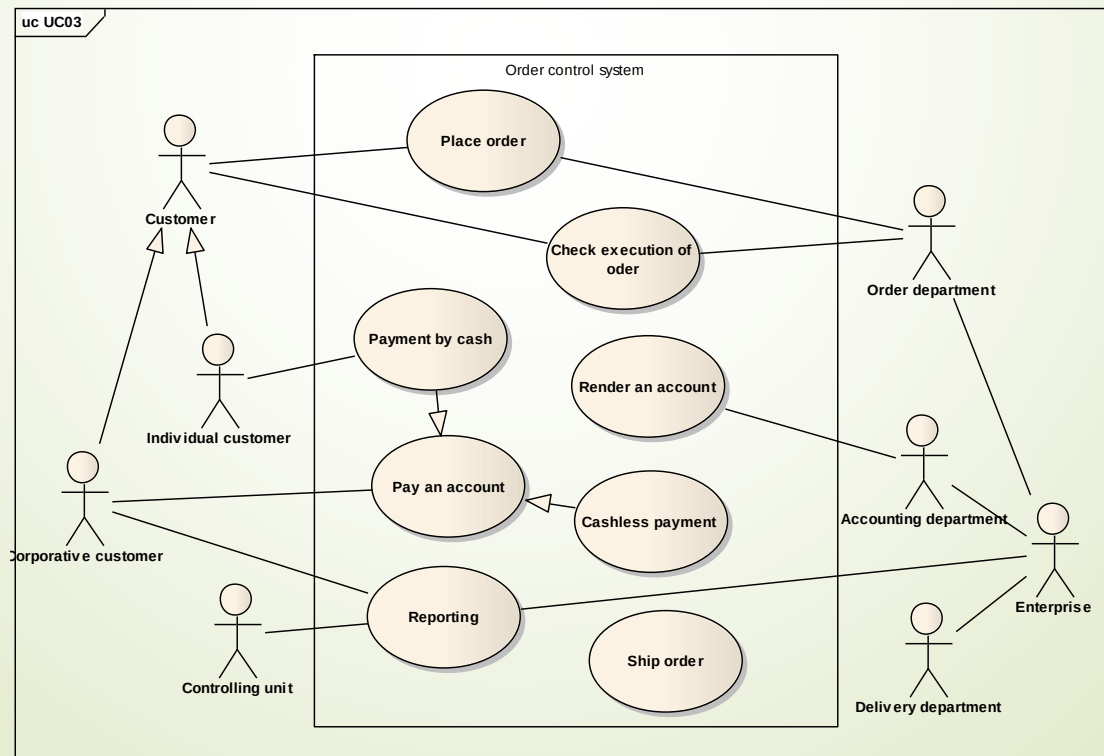


➤ The extend relationship (a dependency with the «extend» stereotype) implies that one use case (the extendable one) implicitly incorporates the behavior of another use case (the extending one) at a point indirectly specified by the extending use case.

- Extend relationships are used to model parts of a use case that the user perceives as optional system behavior.
- Extend relationships are used to model specific sub-flows that are executed only under certain circumstances.

System context modeling

- Context modeling involves drawing an imaginary boundary around the system and identifying the actors located outside that boundary who interact with the system. At this stage, use case diagrams are used to identify the actors and the semantics of their roles.



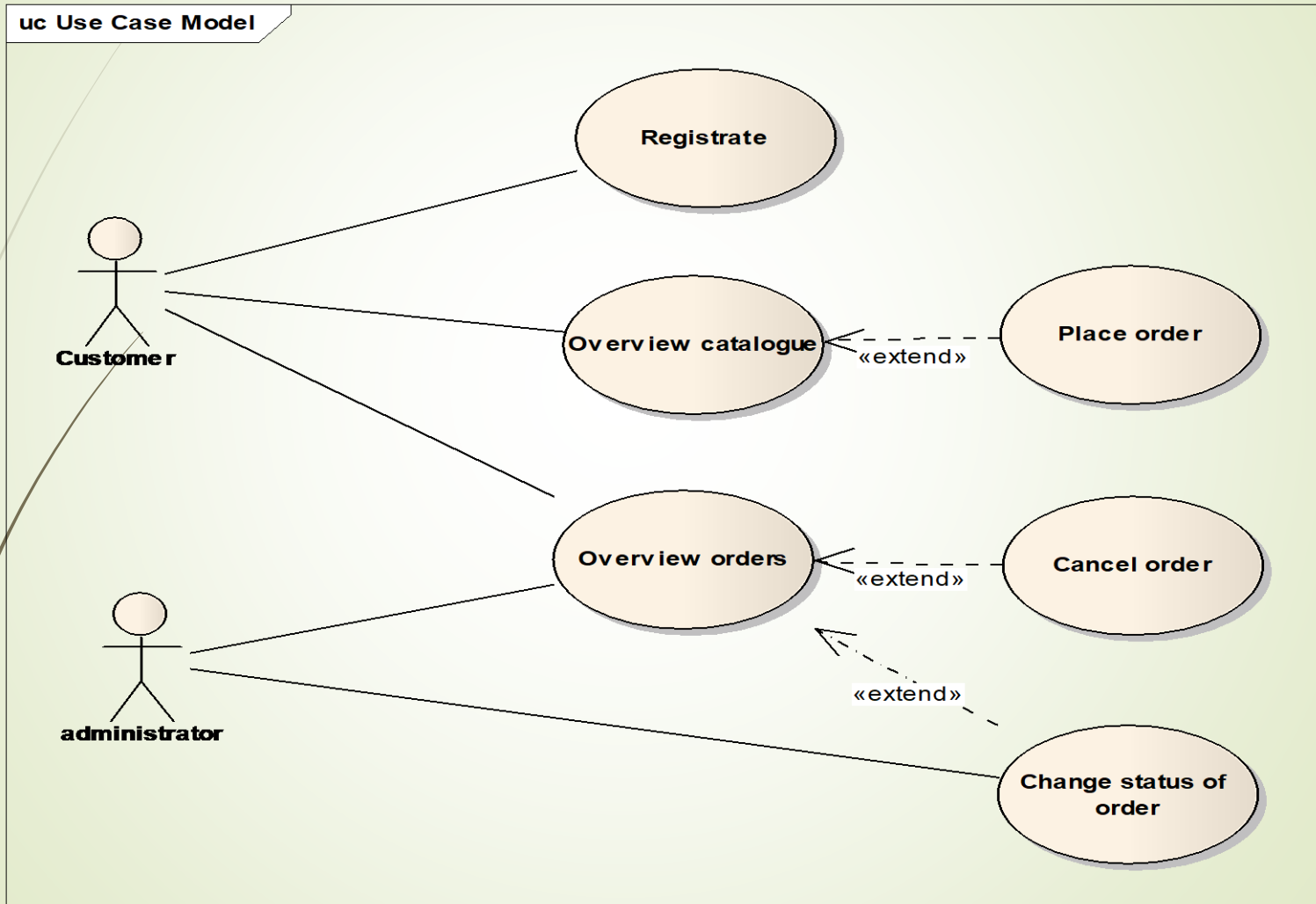
System context modeling

- Identify the actors in the system's environment. Find groups that require the system's participation to perform tasks; groups the system needs to carry out its functions; groups that interact with external software and hardware; and groups that perform auxiliary administration and support functions
- *Organize similar actors using generalization/specialization relationships*
- Enter stereotypes for each actor if it makes understanding easier
- *Place the actors on the use case diagram and define how they relate to the system's use cases*

A good use case

- designates a simple, identifiable atomic behavior of the system or a part thereof
- *isolates common behavior by extracting it from all the use cases that include it*
- identifies variations by placing certain behavior into other use cases that extend it
- *describes the sequence of events in sufficient detail for an outside reader to understand*
- described using a minimal set of scripts specifying its normal and additional semantics

Example: Online store



Example of a Use Case diagram description

Function No.:	UC.01.3
Name:	Processing a candidate for admission to a study program
Acting entity:	1) HR manager 2) Candidate for training 3) Trainer
Description:	Student enrollment. This stage occurs after registration, but not all training candidates undergo the testing stage (this depends on the manager's decision).
System state prior to start:	The client successfully completed the registration procedure (UC.01.1). Some of the training candidates successfully passed the testing stage (UC.01.2).
System state after completion:	The candidate has been successfully enrolled
Normal course:	1. The trainer schedules the training session. 2. The manager records the training date and time in the trainee's file. 3. The manager creates an account for the trainee. 4. The manager prepares the necessary documents and records the document details in the trainee's personal file. 5. The trainee signs all necessary documents.
Alternative current:	No
Exceptions:	No
Includes:	UC.01.3.2 UC.01.3.3
Priority (Critical Important Desirable):	Important
Frequency of use:	Once for each candidate
Special requirements:	No

1. Develop Use Case diagrams for your processes
2. Draft the descriptions for the Use Case diagrams